

Internal representations of numbers

◆ Representations of *integers*:

- *unsigned representation*: only for natural numbers
- *signed representations (codes)*: for integers with sign
 - direct code
 - inverse code (one's complement)
 - complementary code (two's complement)

◆ Representations of *real numbers*:

- *fixed-point representation*
- *floating-point representation*



- | bit $n-1$ | bit $n-2$ | ... | bit 1 | bit 0 |
|-----------|-----------|-----|-------|-------|
| | | | | |

R1) *Addition and subtraction* on n bits: both terms and the result are represented on n bits.

R2) *Multiplication* on n bits: both factors represented on n bits and the product represented on $2 \cdot n$ bits.

R3) *Division* on n bits: *dividend* represented on $2 \cdot n$ bits, the *divisor* on n bits.

This operation provides a *quotient* and a *remainder* represented both on n bits.

R4) In case of *overflow* the most significant bits are lost.

Unsigned representation

- the *natural number* is converted into binary and then the bits (n) from the memory location are filled with the corresponding binary digits (aligned to the right), eventually the leftmost bits are filled with insignificant zeros.
- min: $0 \dots 0_{(2)} = 0$
- max: $1 \dots 1_{(2)} = 2^n - 1$
- intervals of values:

$$n = 8 \quad [0 , 255]$$

$$n = 16 \quad [0 , 65535]$$

$$n = 32 \quad [0 , 4\,294\,967\,295]$$

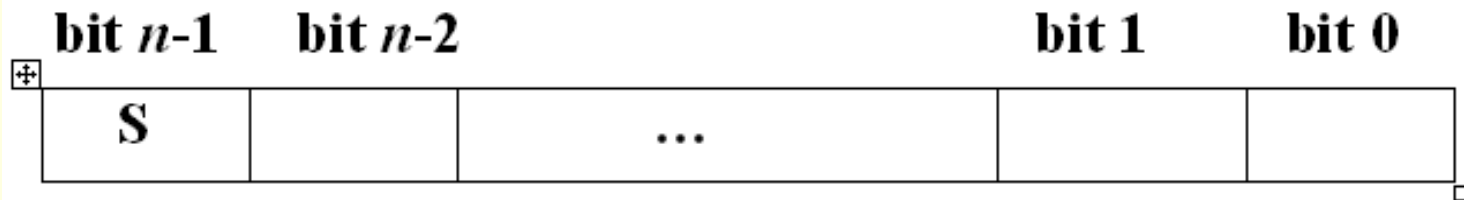
$$n = 64 \quad [0 , 18\,446\,824\,753\,389\,551\,615]$$

Examples of representations and operations on 8 bits

$ \begin{array}{r} 18+ \\ 18 \\ \hline 36 \end{array} $	$ \begin{array}{r} 12_{(16)}+ \\ 12_{(16)} \\ \hline 24_{(16)} \end{array} $	$ \begin{array}{r} 00010010_{(2)}+ \\ 00010010_{(2)} \\ \hline 00100100_{(2)} \end{array} $	$ \begin{array}{r} 243- \\ 18 \\ \hline 225 \end{array} $	$ \begin{array}{r} F3_{(16)}- \\ 12_{(16)} \\ \hline E1_{(16)} \end{array} $	$ \begin{array}{r} 11110011_{(2)}- \\ 00010010_{(2)} \\ \hline 11100001_{(2)} \end{array} $
$ \begin{array}{r} 243+ \\ 18 \\ \hline 261 \end{array} $	$ \begin{array}{r} F3_{(16)}+ \\ 12_{(16)} \\ \hline 05_{(16)} \end{array} $	$ \begin{array}{r} 11110011_{(2)}+ \\ 00010010_{(2)} \\ \hline 100000101_{(2)} \end{array} $! overflow at addition!	$ \begin{array}{r} 18- \\ 243 \\ \hline -225 \end{array} $	$ \begin{array}{r} 12_{(16)}- \\ F3_{(16)} \\ \hline 1F_{(16)} \end{array} $	$ \begin{array}{r} 00010010_{(2)}- \\ 11110011_{(2)} \\ \hline 00011111_{(2)} \end{array} $! overflow at subtraction !

Signed representations – codes (direct, inverse, complementary)

- dimensions of the memory location $n \in \{8, 16, 32, 64\}$ bits
- the most significant bit (bit $n-1$) is used for the sign
 $S=0$ for *positive* numbers
 $S=1$ for *negative* numbers
- for the magnitude of the number $n-1$ bits are used.



Intervals of values for integers: $[-2^{n-1}+1, 2^{n-1}-1]$

$n=8$ $[-127, 127]$

$n=16$ $[-32767, 32767]$

$n=32$ $[-2\,147\,483\,647, 2\,147\,483\,647]$

$n=64$ $[-9\,223\,412\,376\,694\,775\,807, +9\,223\,412\,376\,694\,775\,807]$

Direct code

Let x be an integer, $|x| < 2^{n-1}$. The *direct code* of x is defined as follows:

$$[x]_{\text{dir}} = \begin{cases} x_{(2)} & , \text{if } x \geq 0 \\ 2^{n-1} + |x| & , \text{if } x \leq 0 \end{cases}$$

- for a *positive* number the bits from the memory location are filled with the digits from the binary representation, aligned to the right, the sign bit is 0;
- for a *negative* number: its absolute value is represented and then the sign bit is set to 1 (equivalent with the operation: $10...0_{(2)} + |x|$)
- *disadvantage*: there are 2 different *direct codes* for 0:

$$\begin{array}{lcl} & \text{S} & \\ [+0]_{\text{dir}} & : & |0|0....0| \quad \text{and} \\ [-0]_{\text{dir}} & : & |1|0....0| \end{array}$$

Inverse code (one's complement)

Let x be an integer, $|x| < 2^{n-1}$. The *inverse code* of x is defined as follows:

$$[x]_{\text{inv}} = \begin{cases} x_{(2)} & , \text{if } x \geq 0 \\ 2^n - 1 - |x| & , \text{if } x \leq 0 \end{cases}$$

- the *inverse code* of a *positive* number is the same as the *direct code*
- for a *negative* number: its absolute value is represented and then the values of all bits are inversed (*complemented to 1*):
equivalent with the operation: $11...1_{(2)} - |x_{(2)}|$;
- *disadvantage*: there are 2 different *inverse codes* for 0:

$$\begin{array}{l} \text{S} \\ [+0]_{\text{inv}}: \quad |0|0...0| \quad \text{and} \\ [-0]_{\text{inv}}: \quad |1|1...1| \end{array}$$

Complementary code (two's complement)

Let x be an integer, $|x| < 2^{n-1}$. The *complementary code* of x is defined as follows:

$$[x]_{\text{compl}} = \begin{cases} x_{(2)} & , \text{if } x \geq 0 \\ 2^n - |x_{(2)}| & , \text{if } x < 0 \end{cases}$$

- for a *positive* number x , $[x]_{\text{compl}} = [x]_{\text{inv}} = [x]_{\text{dir}}$
- for a *negative* number: its absolute value is represented, then from right to left, beginning with the rightmost bit, all the bits with value 0 and the first bit with the value 1 remain unchanged and all the other bits to the left are inversed.
- if $x < 0$, then $[x]_{\text{compl}} = [x]_{\text{inv}} + 1$ S
- *advantage*: there is a *unique code for 0*: $[0]_{\text{compl}}: |0|0...0|$
- there is a configuration of bits which does not correspond to a number: S
 $|1|0...0|$

Examples of codes on 8 bits

$$\begin{aligned} 100 &= 1100100_{(2)} & S \\ [100]_{\text{dir}} &= [100]_{\text{inv}} = [100]_{\text{compl}} = |0|1100100| \\ [-100]_{\text{dir}} &= |1|1100100| \\ [-100]_{\text{inv}} &= |1|0011011| \\ [-100]_{\text{compl}} &= |1|0011100| \end{aligned}$$

$$\begin{aligned} 40 &= 101000_{(2)} & S \\ [40]_{\text{dir}} &= [40]_{\text{inv}} = [40]_{\text{compl}} = |0|0101000| \\ [-40]_{\text{dir}} &= |1|0101000| \\ [-40]_{\text{inv}} &= |1|1010111| \\ [-40]_{\text{compl}} &= |1|1011000| \end{aligned}$$

Addition and subtraction of integers in complementary code

The complementary codes of the *sum* and *difference* of the integers $x, y \geq 0$:

$$[X+y]_{\text{compl}} = [X]_{\text{compl}} \oplus [y]_{\text{compl}}$$

$$[X-y]_{\text{compl}} = [X]_{\text{compl}} \oplus [-y]_{\text{compl}}$$

The algebraic sum: $\oplus$, on n bits of the complementary codes (c_1 and c_2) of two integers x_1 and x_2 is defined as follows:

$$c_1 \oplus c_2 = \begin{cases} c_1 + c_2 & \text{if } c_1 + c_2 < 2^n \\ c_1 + c_2 - 2^n & \text{if } c_1 + c_2 \geq 2^n \end{cases}$$

where $c_1 + c_2$ is the binary sum on n bits.

Rules

r1: if x_1 and x_2 have the same sign, but $c_1 \oplus c_2$ has the opposite sign $\Rightarrow$ **overflow**.

r2: if in the result of $c_1 \oplus c_2$, there is a carry digit outside the representation space, this will be eliminated (the second branch of the above definition).
If there is no overflow we interpret the content of the memory location (n bits) as a complementary code of an integer number.

Examples

$$\begin{array}{rcl}
 [40+40]_{\text{compl}} & = & [40]_{\text{compl}} \oplus [40]_{\text{compl}} \\
 & & \text{-----} \\
 [80]_{\text{compl}} & = & 0|1010000 - \text{correct result}
 \end{array}$$

S
0|0101000⊕
0|0101000

$$\begin{array}{rcl}
 [100+100]_{\text{compl}} & = & [100]_{\text{compl}} \oplus [100]_{\text{compl}} \\
 & & \text{-----} \\
 [200]_{\text{compl}} & = & 1|1001000 - \text{overflow (r1)}
 \end{array}$$

S
0|1100100⊕
0|1100100

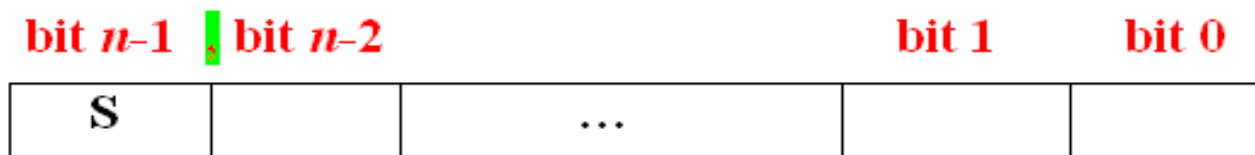
$$\begin{array}{rcl}
 [100-40]_{\text{compl}} & = & [100]_{\text{compl}} \oplus [-40]_{\text{compl}} \\
 & & \text{-----} \\
 [60]_{\text{compl}} & = & 10|0111100 - \text{correct result (r2)}
 \end{array}$$

S
0|1100100⊕
1|1011000

Subunitary convention

— Let $x \in \mathbb{R}$, $|x| < 1$, with at most $n-1$ binary digits after the decimal point.

Memory location: $n \in \{8, 16, 32, 64\}$ bits



direct code of x :

$$[x]_{\text{dir}} = \begin{cases} x_{(2)} & , \text{if } x \geq 0 \\ 1 + |x_{(2)}| & , \text{if } x < 0 \end{cases}$$

inverse code (complement to 1) of x :

$$[x]_{\text{inv}} = \begin{cases} x_{(2)} & , \text{if } x \geq 0 \\ 2 - 2^{-n+1} - |x_{(2)}| & , \text{if } x < 0 \end{cases}$$

complementary code (complement to 2) of x :

$$[x]_{\text{compl}} = \begin{cases} x_{(2)} & , \text{if } x \geq 0 \\ 2 - |x| & , \text{if } x < 0 \end{cases}$$

Codes for signed subunitary numbers

- for a *positive number*:
 - the number is converted into binary
 - the sign bit is set to 0, then,
 - beginning with the bit $n-2$, from left to the right, the bits are filled with the corresponding binary digits.
- the same practical rules for obtaining the codes for *negative numbers* as for integers are applied.
- if $x \geq 0$, then $[x]_{\text{compl}} = [x]_{\text{inv}} = [x]_{\text{dir}}$
- if $x < 0$, then $[x]_{\text{compl}} = [x]_{\text{inv}} + 2^{-n+1}$

Examples:

$$13/16 = 0, D_{(16)} = 0, 1101_{(2)}$$

$$[13/16]_{\text{dir}} = [13/16]_{\text{inv}} = [13/16]_{\text{compl}} = \overset{s}{|0|}1101000|$$

$$[-13/16]_{\text{dir}} = |1|1101000|$$

$$[-13/16]_{\text{inv}} = |1|0010111|$$

$$[-13/16]_{\text{compl}} = |1|0011000|$$

$$0,25 = 0,01_{(2)} \quad \overset{s}{|}$$

$$[0,25]_{\text{compl}} = |0|0100000|$$

$$[-0,25]_{\text{compl}} = |1|1100000|$$

Addition and subtraction of subunitary numbers in complementary code

The complementary codes for the sum and difference of two subunitary positive numbers x, y :

$$[x+y]_{\text{compl}} = [x]_{\text{compl}} \oplus [y]_{\text{compl}}$$

$$[x-y]_{\text{compl}} = [x]_{\text{compl}} \oplus [-y]_{\text{compl}}$$

The algebraic sum $\oplus$ on n bits of the complementary codes (c_1 and c_2) of two subunitary numbers, x_1 and x_2 , is defined as follows.

! the virtual position of the decimal point is after the leftmost bit.

$$c_1 \oplus c_2 = \begin{cases} c_1 + c_2, & \text{if } c_1 + c_2 < 10_{(2)} = 2 \\ c_1 + c_2 - 10_{(2)}, & \text{if } c_1 + c_2 \geq 10_{(2)} = 2 \end{cases}$$

where $c_1 + c_2$ is the binary sum on n bits

Rules

r1: if x_1, x_2 have the same sign, but $c_1 \oplus c_2$ has the opposite sign $\Rightarrow$ **overflow**.

r2: if in the result of $c_1 \oplus c_2$, there is a carry digit outside the representation space, this will be eliminated (the second branch of the above definition).

If there is no overflow we interpret the content of the memory location (n bits) as a complementary code of a subunitary number.

Examples

addition and subtraction of subunitary numbers

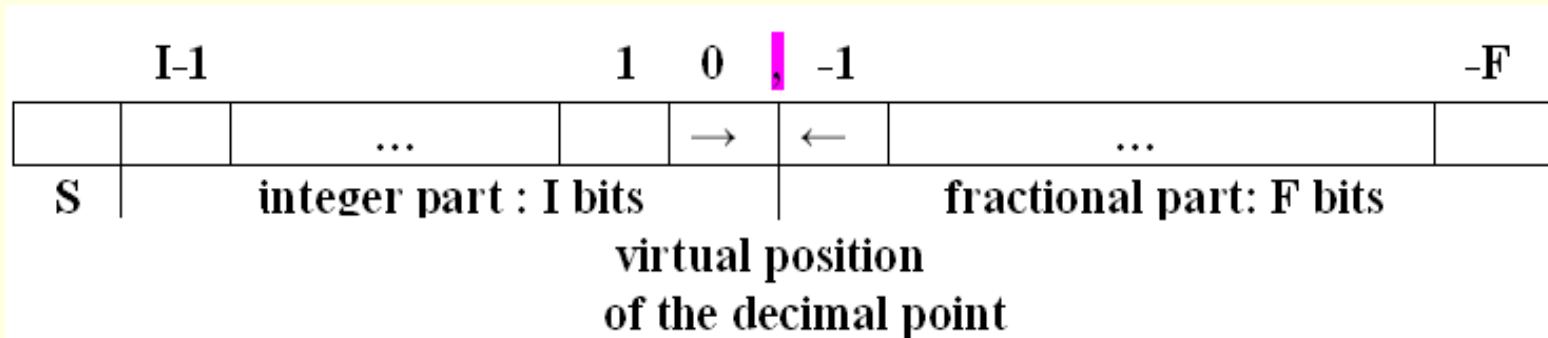
$$\begin{array}{rcl}
 [0,25+0,25]_{\text{compl}} & = & [0,25]_{\text{compl}} \oplus [0,25]_{\text{compl}} \\
 & & \begin{array}{r} s_1 \\ 0|0100000 \oplus \\ 0|0100000 \\ \hline 0|1000000 \end{array} \quad \text{correct result}
 \end{array}$$

$$\begin{array}{rcl}
 [0,25+13/16]_{\text{compl}} & = & [0,25]_{\text{compl}} \oplus [13/16]_{\text{compl}} \\
 & & \begin{array}{r} s_1 \\ 0|0100000 \oplus \\ 0|1101000 \\ \hline 1|0001000 \end{array} \quad \text{- overflow (r1)} \\
 & & \text{!!positive operands, negative sum !!}
 \end{array}$$

$$\begin{array}{rcl}
 [13/16-0,25]_{\text{compl}} & = & [13/16]_{\text{compl}} \oplus [-0,25=-4/16]_{\text{compl}} \\
 & & \begin{array}{r} s_1 \\ 0|1101000 \oplus \\ 1|1100000 \\ \hline 1|01001000 \end{array} \\
 [0,5625=9/16]_{\text{compl}} & = & 1|01001000 \quad \text{correct result (r2)}
 \end{array}$$

Fixed-point representation of real numbers

- Dimensions of memory location: $n = 16, 32, 64$ bits
- 3 zones of the memory location with predefined dimensions (1, I, F): $1 + I + F = n$ bits
- the most significant bit (S), position $n-1$, is the sign bit with the values: 0 for positive numbers and 1 for negative numbers;
- the decimal point has a fixed position, a virtual one, separating the integer part from the fractional one;



Fixed-point representation (contd.)

◆ the integer part (I bits)

- memorizes (aligned to the right relative to the virtual position of the decimal point) the digits of the absolute integer value of the number converted into binary;
- if $I >$ the number of digits of the binary representation of the absolute integer value of the number, the remaining bits to the left are filled with 0.
- if $I <$ the number of digits of the binary representation of the absolute value of the number, then the most significant digits of the integer part are lost **(!! disadvantage)**.

◆ the fractional part (F bits)

- memorizes (aligned to the left relative to the virtual position of the decimal point) the digits of the fractional part
- if $F >$ the number of binary digits of the fractional part then the remaining digits to the right are filled with 0.
- if $F <$ the number of binary digits of the fractional part then the least significant digits of the fractional part are lost.

Fixed-point representation (contd.)

A nonzero real number x is represented in fixed-point convention on $n=1+I+F$ bits if: $2^{-F} \leq |x| \leq 2^I - 2^{-F}$

The representation of the *minimum absolute value* 2^{-F} is:

S	I bits			,	F bits		
0	0	...	0	0	...	0	1

The representation of the *maximum absolute value* $2^I - 2^{-F}$ is:

S	I bits			,	F bits		
0	1	...	1	1	...	1	1

Remark: $2^I = 10...0,0...00_{(2)}$ -
 $2^{-F} = 0,0...01_{(2)}$

 $1...1,1...11_{(2)}$

Example 1

Represent in fixed point convention on 16 bits, I=6 bits, F=9 bits
the number: – 29,21.

Conversion into binary:

- *integer part*: the number is written as a sum of powers of 2:

$$29 = 16 + 8 + 4 + 1 = 2^4 + 2^3 + 2^2 + 2^0 = \mathbf{11101}_{(2)}$$

- *fractional part*: successive multiplications by the destination base 2:

9 multiplications by 2 (F=9 bits)

$$0,21 * 2 = \underline{0},42 \quad 0,42 * 2 = \underline{0},84 \quad 0,84 * 2 = \underline{1},68 \quad 0,68 * 2 = \underline{1},36$$

$$0,36 * 2 = \underline{0},72 \quad 0,72 * 2 = \underline{1},44 \quad 0,44 * 2 = \underline{0},88 \quad 0,88 * 2 = \underline{1},76$$

$$0,76 * 2 = \underline{1},52$$

$$0,21 = 0, \mathbf{001101011}_{(2)}$$

$$- 29,21 = - \mathbf{11101}, \mathbf{001101011}_{(2)}$$

Representation:

S	6 bits						9 bits									
1	0	1	1	1	0	1	0	0	1	1	0	1	0	1	1	= BA6B

location's content in hexa

Example 2

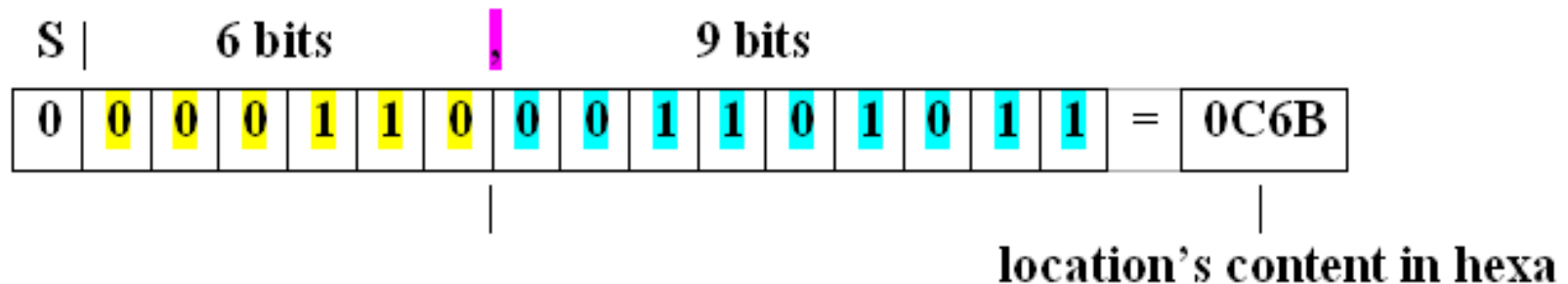
Represent in fixed point convention on 16 bits, I=6 bits, F=9 bits,
the number: + 70,21.

$$70 = 64 + 4 + 2 = 2^6 + 2^2 + 2^1 = 1000110_{(2)},$$

$$70,21 = 1000110,001101011_{(2)}$$

Remark:

The integer part does not fit in 6 bits, the most significant bit is lost: 1.



Floating point representation of real numbers

- ♦ used to represent very large and very small numbers with a high precision
- ♦ advantage:
 - if there is an overflow, then the *least significant digits are lost*

Any real number x can be written as: $x = \pm 0, m \cdot b^e$ where:
m - mantissa ; **b** - numeration base; **e** - exponent

Examples:

$$1234,5678 = 0,12345678 * 10^4$$

$$0,004371 = 0,4371 * 10^{-2}$$

$$1101,0011_{(2)} = 0,11010011_{(2)} * 2^4$$

Definitions

Def 1: A real number x is written *with a subunitary mantissa and an exponent* of base b if $x = \pm 0,m \cdot b^e$

Def 2: A real number x , $x \neq 0$, is written *with a subunitary normalized mantissa*, if x is written with a subunitary mantissa and an exponent of base b and $\frac{1}{b} \leq m < 1$.

Ex: $0,1010011_{(2)} * 2^4$ - subunitary normalized mantissa
 $0,0011101_{(2)} * 2^{-4}$ - subunitary not normalized mantissa $\Rightarrow$
 $\Rightarrow 0,11101_{(2)} * 2^{-6}$ - mantissa was normalized

Def 3 : A real number x , $x \neq 0$, is written with a $1 < \text{mantissa} < 2$, if $x = \pm 1,m \cdot 2^e$

Ex: $1,010011_{(2)} * 2^3$ $< 1 \text{ mantissa} < 2$, **1** is the hidden bit

Def 4: A real number x is represented in floating point notation if it is used the binary representation with exponent and subunitary normalized mantissa or $1 < \text{mantissa} < 2$.

Floating point representation (contd.)

Memory location:

	E bits	M bits
S	$c = e + q$	m

, virtual position of the decimal point

where:

S : *sign bit* (0 for positive numbers, 1 for negative numbers)

$c = e$ (*exponent*) + q (*bias*)

q : *bias* (a constant of the computer)

e : the *exponent* from the binary representation with mantissa and exponent

m : *mantissa* from the binary representation with mantissa and exponent

The precision is provided by the *min* and *max* values of the exponent:

$$0 \leq c = (e + q) \leq 2^E - 1 \quad \Rightarrow \quad -q \leq e \leq 2^E - 1 - q$$

Remarks:

1. If $1 < \text{mantissa} < 2$ the digit 1 from the integer part is not represented internally (is hidden), but it will be used in the operations with the representations.
2. Computers use now $1 < \text{mantissa} < 2$.

IEEE Standards: $1 < \text{mantissa} < 2$

Standard IEEE 754 - Single precision

32 bits

E=8 bits, M=23 bits

	8 bits	23 bits
S	$c = e + 127$	m

$$q = 127 = 2^7 - 1 = 2^{E-1} - 1, \quad -127 \leq e \leq 128$$

Standard IEEE 754 - Double precision

64 bits

E=11bits, M=52 bits

	11 bits	52 bits
S	$c = e + 1023$	m

$$q = 1023 = 2^{10} - 1 = 2^{E-1} - 1, \quad -1023 \leq e \leq 1024$$

Special values

Value	Representation		
	S	c	m
0_+	0	0...0	0...0
0_-	1	0...0	0...0
$-\text{inf}$	1	1...1	0...0
$+\text{inf}$	0	1...1	0...0
NaN (not a number)	1 sau 0	1...1	Nonzero value

Floating point representation (contd.)

1. The smallest positive number that can be represented in single precision has the representation:

S	c	m
0	00000001	000000000000000000000000

$$c=1 \Rightarrow e + 127(q) = 1 \Rightarrow e = -126$$

Using the hidden bit the value of the mantissa is 1.0 and:

$$\min = 1.0 * 2^{-126} = 2^{-126}$$

2. The largest positive number that can be represented in single precision has the representation:

S	c	m
0	11111110	111111111111111111111111

$$c=11111110_{(2)} = 2^8 - 2 = 254 \text{ (the biggest value of } c \text{ for a valid number)}$$

$$\Rightarrow e + 127(q) = 254 \Rightarrow e = 127$$

Using the hidden bit, the value of the mantissa is:

$$1,111111111111111111111111_{(2)} = 2 - 0,000000000000000000000001_{(2)} = 2 - 2^{-23}$$

$$\max = (2 - 2^{-23}) * 2^{127}$$

Min and max values

Precision	Binary absolute value	Decimal absolute value
Single	$\min = 2^{-126}$ $\max = (2 - 2^{-23}) * 2^{127}$	$\min \approx 10^{-38}$ $\max \approx 10^{38}$
Double	$\min = 2^{-1022}$ $\max = (2 - 2^{-52}) * 2^{1023}$	$\min \approx 10^{-308}$ $\max \approx 10^{308}$

Example: Represent in floating point notation, single precision, with mantissa>1, the number 2530,41.

Conversion of the integer part: successive divisions by 8

$$2530 = 4742_{(8)} = 100\ 111\ 100\ 010_{(2)} = 1,00111100010_{(2)} * 2^{11}$$

The number was written with *exponent* and a *mantissa* >1,

1 is the **hidden bit** that is not represented internally

$$\Rightarrow e=11 \text{ and } c = 127+11=138 = 128+8+2 = 2^7+2^3+2^1 = 10001010_{(2)}$$

Conversion of the fractional part: successive multiplications by 8

- there are 11 digits in mantissa from the integer part
- we need 12 more binary digits obtained from the fractional part:

12 binary digits are covered by 4 octal digits: 4 multiplications by 8

$$0,41 * 8 = 3,28, \quad 0,28 * 8 = 2,24, \quad 0,24 * 8 = 1,92, \quad 0,92 * 8 = 7,36$$

$$0,41 = 0,3217_{(8)} = 0,011\ 010\ 001\ 111_{(2)} \text{ (rapid conversion)}$$

$$2530,41 = 1,00111100010011010001111_{(2)} * 2^{11}$$

Representation on 32 bits:

S	c (8bits)	m (23 bits)	memory content in hexa
0	10001010	00111100010011010001111	= 451E268F